

Lattice.addIntent and Cache Usage

A verbal algorithm guide to the incremental CVA–FCA insertion implemented in `src/cvafca/Lattice.scala`.

In one sentence: `addIntent` treats an incoming instance as a singleton learned intent, searches upward from nearby leaf concepts for the most specific compatible generator, locally creates or reuses concepts, rewires symbolic containment edges, and finally adds the instance to accepting ancestors and refits their CVA models.

1. Mental model

Each lattice node contains a symbolic extent (instance symbols), a vector extent, a learned CVA model (common vector, projection, and classification radius), and links to parents and children.

parent → child
larger symbolic extent → smaller symbolic extent
more general → more specific

Learned CVA classification is used to search for compatible concepts. Strict symbolic set inclusion is used for graph edges. Keeping these responsibilities separate prevents a non-transitive learned classifier from creating cycles in the lattice topology.

2. Algorithm, step by step

Step 1 — Handle an empty lattice

If no nodes exist, the method creates a singleton node containing the incoming instance and returns it.

Step 2 — Create a temporary singleton target

The incoming instance x is fitted as `target = concept({x})`. This temporary node represents the concept being inserted. It is placed in the lattice only if an equivalent existing node cannot be reused.

Step 3 — Choose a bounded bottom neighborhood

The method samples up to 32 leaf nodes (nodes without children), sorts them by CVA distance to the incoming vector, and retains at most the closest 8 as search starts. The implementation deliberately caps work:

```
maxRecursionDepth      = 1
maxSearchVisits         = 24
maxBottomSamples       = 32
maxBottomStarts        = 8
maxChildrenPerGenerator = 4
```

This keeps insertion cost local and predictable, but means the globally best generator can be missed in a large lattice.

Step 4 — Search upward for a generator

`maximalConcept` performs a bottom-up, best-first search, prioritizing nodes closest to the incoming vector.

A node is compatible when:

1. Its learned model classifies enough of the target extent. For a singleton target, this essentially asks whether it classifies the incoming instance.
2. It is coherent with the incoming item: the item is close to the concept, and the singleton target also sees every member of that concept as sufficiently close.

Once the first compatible node on a path is reached, its less-specific ancestors are not explored. Among compatible candidates, the node with the smallest extent is selected; ID breaks ties deterministically.

Step 5 — Record accepting ancestors

If a generator is found, `ancestorClosure` collects it and its ancestors, up to the visit limit. Their IDs form `acceptedIds`. These nodes will receive the instance at the end.

During structural work, `effectiveSymbols` pretends accepted nodes already contain the new symbol. This permits future symbolic ordering to be calculated without changing any learned model mid-search.

Step 6 — If no generator exists

There is no artificial universal node. If nothing classifies the singleton, the method examines coherent root nodes, chooses the closest, creates a learned concept over `root.instances ∪ {x}`, and links it above the old root when the symbolic order permits. It then searches again.

Step 7 — Insert recursively in the local neighborhood

`addIntentRecursively` has three main outcomes:

- **No generator:** materialize the target as a new node and add it to the bottom frontier.
- **Equivalent generator:** reuse that node instead of creating a duplicate.
- **Distinct generator:** inspect up to four closest children. Reuse compatible children or construct candidates fitted over `child.instances ∪ target.instances`. Recursion is limited to depth 1.

"Intersection" here refers to learned intent semantics; operationally, its CVA model is fitted to the union of object extents.

Step 8 — Remove redundant prospective parents

Candidate parents are compared with directional learned-containment tests. If one candidate makes another redundant, only the more appropriate local cover candidate is retained.

Step 9 — Materialize or reuse the target

`materialize` searches locally for an existing mutually equivalent concept. It reuses that concept when found; otherwise it inserts the fitted target into `nodeMap`.

Step 10 — Rewire local edges

Edges between the generator, new concept, and nearby concepts are created according to strict symbolic inclusion. When the new concept belongs between two nodes, the old direct edge is removed:

before: generator → old child
after: generator → new concept → old child

`link` also maintains the bottom frontier: parents with children are removed from it, while childless nodes are included.

Step 11 — Mutate and refit only after structure is stable

After local insertion finishes, the instance is added to every node in `acceptedIds`, and each changed CVA model is refitted. Delaying refits ensures every search and edge decision in one insertion observes a stable snapshot of the old models.

3. Cache usage

Every cache is created inside `addIntent`. It exists for one insertion only and is discarded afterward.

Cache	Key	Stored value	Role
<code>targetCache</code>	Symbol set	Target/canonical node	Avoids fitting the same symbolic extent repeatedly. After equivalent-node reuse, the entry points to the reused canonical node.
<code>incomingDistanceCache</code>	Node ID	Distance to incoming vector	Avoids repeated tensor projection and distance computation during sampling, ordering, searching, and coherence checks. NaN becomes positive infinity.
<code>coherenceCache</code>	Node ID	Boolean	Avoids repeating concept-to-item and singleton-to-members proximity checks, including scans over a concept's extent.
<code>subsetCache</code>	Ordered pair of node IDs	Boolean	Caches directional learned coverage: whether the left model classifies enough instances of the right node.

Cache	Key	Stored value	Role
activeCalls	Target symbols + generator ID	Presence marker	A recursion guard. Re-entering an active target/generator pair stops recursion and materializes the target.

Important cache details

- `intentSubset(left, right)` is learned semantic coverage, not ordinary Scala set inclusion, and it is directional.
- `targetCache` assumes a symbol uniquely identifies an instance/vector because the key contains symbols only.
- Distance, coherence, and subset entries remain valid because existing node models are not refitted until structural insertion is complete.
- The caches are not valid across insertions, which is why none is stored as a long-lived field of `Lattice`.

4. Behavior and trade-offs

- Search is distance-guided and locally bounded.
- Equivalent concepts found in the searched neighborhood are reused.
- Expensive learned comparisons are evaluated once per relevant key.
- Topology remains cycle-safe through strict symbolic inclusion.
- The globally best generator or an equivalent concept outside the bounded neighborhood may be missed.
- Only local cover edges are updated; the complete Hasse diagram is not rebuilt.
- Because learned classification may lack exact closure, antisymmetry, and transitivity, this is an approximate learned FCA lattice rather than a mathematically exact formal concept lattice.

Core reason for the cache design: one insertion repeatedly asks the same expensive questions while traversing overlapping local paths. Memoization makes those questions cheap, and delayed refitting guarantees their answers stay consistent until the insertion finishes.